

Introducción a la programación

Complemento a 2

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2025-II

UACM

Universidad Autónoma
de la Ciudad de México

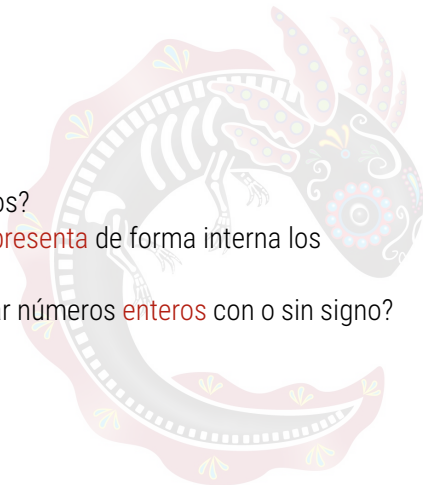
NADA HUMANO ME ES AJENO

Preguntas...

¿Cuál es la **unidad de medida** de los datos?

¿Sabes como es que la computadora **representa** de forma interna los **números**?

¿Crees que exista diferencia al almacenar números **enteros** con o sin signo?



Vectores binarios

Un solo **dígito binario** se denomina **bit**. Para representar números, **agrupamos** los bits en **secuencias** más grandes las cuales denominamos **vectores binarios**.

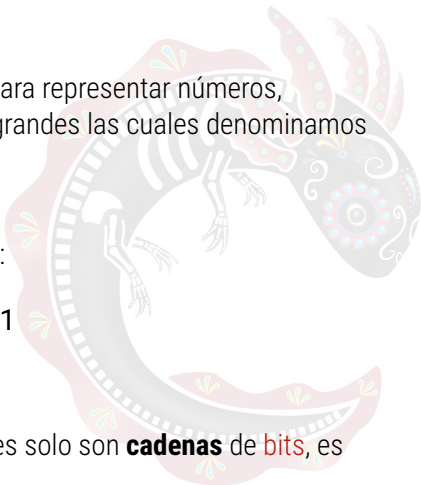
Ejemplo:

Consideramos el siguiente vector de bits:

1101111

El vector tiene **7 bits**.

Es importante mencionar que los vectores solo son **cadenas** de **bits**, es decir, no tienen **ningun significado**.

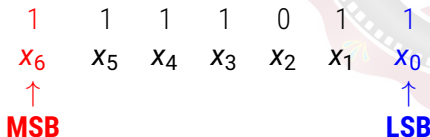


Bits más y menos significativos

Hacen referencia a los bits que tienen **mayor** y **menor** valor. Para identificarlos, se leen los bits de **derecha a izquierda**, asignando a cada bit un **índice**. El bit más a la **derecha** (bit cero o x_0) se denomina el **menos significativo (LSB)**, mientras que el bit más a la **izquierda** (bit $n - 1$ o x_{n-1}) es el **más significativo (MSB)**.

Ejemplo:

Sea el vector binario **1111011**, identifique el bit mas significativo y el menos significativo:



Conjuntos especiales de vectores binarios

Un vector de **cuatro bits** es nombrado **nibble**.

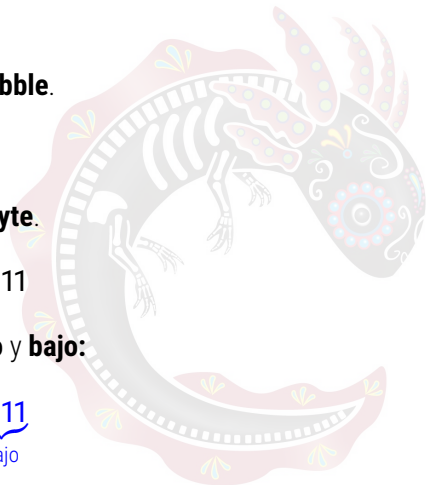
1001

Un vector de **ocho bits** se le denomina **byte**.

10101111

Un byte se compone de **dos nibbles**, **alto** y **bajo**:

10101111
alto bajo



Múltiplos del byte

El byte se utiliza como **medida de almacenamiento** de información, por lo que los **múltiplos** del byte son los siguientes:

Nombre	Descripción	Valor
Byte (B)	Conjunto de 8 bits	8 bits
Kilobyte (KB)	Conjunto de 1024 bytes	1024×8 bits
Megabyte (MB)	Conjunto de 1024 KB	$1024^2 \times 8$ bits
Gigabyte (GB)	Conjunto de 1024 MB	$1024^3 \times 8$ bits
Terabyte (TB)	Conjunto de 1024 GB	$1024^4 \times 8$ bits
Petabyte (PB)	Conjunto de 1024 TB	$1024^5 \times 8$ bits
Exabyte (EB)	Conjunto de 1024 PB	$1024^6 \times 8$ bits

Información

La **información** es el conjunto de **datos organizados** y **procesados** que constituyen **mensajes, instrucciones, operaciones, funciones** y cualquier tipo de actividad que tenga lugar en relación con la computadora.

La información se puede **clasificar** como

- **Datos numéricos:** generalmente combinaciones de **dígitos** del 0 al 9.
- **Datos alfabéticos:** compuestos sólo por **letras**.
- **Datos alfanuméricos:** combinación de los **dos anteriores**.

Representación de la información

Dependiendo del tipo de **información**, existen diferentes formas de **representar datos** en una **computadora**:

■ **Números enteros**

- Representación binaria
- Signo magnitud
- Complemento 1
- Complemento 2

■ **Números reales** (con punto decimal)

- Estándar IEEE-754

■ **Caracteres alfanuméricos**

- ASCII
- UNICODE

Para este curso solo nos enfocaremos en los **números enteros** y **caracteres alfanuméricos**.



Representación binaria

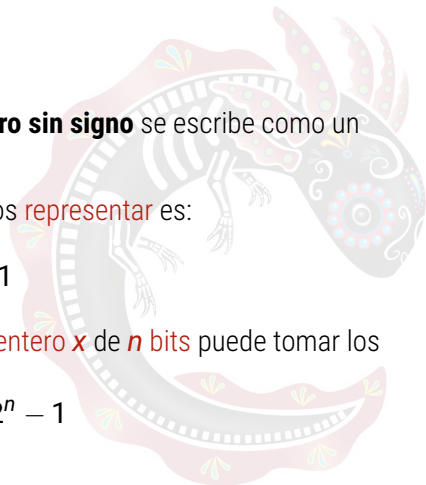
En esta representación, un **número entero sin signo** se escribe como un **número binario** de n bits.

Con n bits, el **valor máximo** que podemos **representar** es:

$$2^n - 1$$

Por lo tanto, en esta representación, un **entero x** de n bits puede tomar los **valores**:

$$0 \leq x \leq 2^n - 1$$



Representación binaria

Ejemplo

Represente el numero 57 como entero sin signo con 8 bits.

- **Verificamos** que el número pueda ser representable con 8 bits:

$$\begin{aligned}2^n - 1 &= 2^8 - 1 \\&= 256 - 1 \\&= 255\end{aligned}$$

Como $57 \leq 255$ entonces podemos representar el 57 con 8 bits.

- **Convertimos** el 57 a binario:

$$57_{10} = 111001_2$$

- Finalmente, **escribimos** el 57 con 8 bits:

$$57 = 00111001$$

Representación signo-magnitud

Sirve para representar **números entero con signo** con n bits asignando el bit **más significativo** para almacenar el **signo** y **el resto** de los bits ($n - 1$) para almacenar la **magnitud**. El **bit de signo** se establece en **1** para un **entero negativo** y **0** para un **entero positivo**. La **magnitud** se escribe como **número binario** con $n - 1$ bits.

En esta representación, un **entero x** de n bits puede tomar los **valores**:

$$-2^{n-1} - 1 \leq x \leq +2^{n-1} - 1$$

En esta representación hay **dos versiones** del **cero**, es decir, ± 0 . Por ejemplo, para 8 bits:

$$00000000 = +0$$

$$10000000 = -0$$

Representación signo-magnitud

Ejemplo:

Represente el ± 27 en formato signo-magnitud con 8 bits:

- Con **8 bits** podemos **representar**:

$$\begin{aligned} -2^{8-1} - 1 \leq x \leq +2^{8-1} - 1 &= -2^7 - 1 \leq x \leq +2^7 - 1 \\ &= -127 \leq x \leq +127 \end{aligned}$$

- **Convertimos** el 27 a **binario**:

$$27_{10} = 11011_2$$

- Finalmente, **escribimos** la magnitud con **7 bits** y agregamos los **bits de signo**:

$$\mathbf{0}0011011 = +27$$

$$\mathbf{1}0011011 = -27$$

Representación complemento a 1

En esta representación, asignamos el bit **más significativo** para almacenar el **signo** y **el resto** de los bits ($n - 1$) para almacenar la **magnitud**. Los **números positivos** se obtienen de la misma forma que en el formato **signo-magnitud** mientras que los **números negativos** se obtienen **intercambiando** **ceros por unos y unos por ceros**.

En esta representación, un **entero x** de **n bits** puede tomar los **valores**:

$$-2^{n-1} - 1 \leq x \leq +2^{n-1} - 1$$

En esta representación también hay **dos versiones** del **cero**, es decir, ± 0 .
Por ejemplo, para 8 bits:

$$00000000 = +0$$

$$10000000 = -0$$

Representación complemento a 1

Ejemplo:

Represente el ± 42 en formato complemento a 1 con 8 bits:

- Al igual que con el formato signo-magnitud, con **8 bits** podemos **representar** el rango $-127 \leq x \leq +127$
- **Convertimos** el 42 a **binario**:

$$42_{10} = 101010_2$$

- El **valor positivo** lo obtenemos escribiendo el número en binario con **7 bits** y **agregando** el **bit de signo**:

$$\mathbf{0}0101010 = +42$$

Representación complemento a 1

- Para obtener el **negativo**, escribimos el número en binario con 7 bits:

0101010

- Después, **intercambiamos** ceros por unos y unos por ceros:

0	1	0	1	0	1	0
↓	↓	↓	↓	↓	↓	↓
1	0	1	0	1	0	1

- Finalmente, **agregamos** el bit de signo:

11010101 = -42

Representación complemento a 2

Es una **extensión** de la representación **complemento a 1**. A diferencia de anterior, el rango es **asimétrico** por lo que elimina el problema de tener **dos representaciones** para el cero. Los **números positivos** se obtienen de la misma forma que complemento a 1 mientras que los **números negativos** se obtienen **realizando primero** la representación **complemento a 1** y **sumando** al resultado **una unidad**.

En esta representación, un **entero x** de **n bits** puede tomar los **valores**:

$$-2^{n-1} \leq x \leq +2^{n-1} - 1$$

Representación complemento a 2

Ejemplo:

Escriba el ± 20 en representación complemento a 2 con 8 bits:

- En este formato, con **8 bits** podemos **representar** el rango $-128 \leq x \leq +127$
- El **positivo** lo obtenemos de la **misma forma** que en **complemento a 1**:

$$00010100 = +20$$

- El **negativo** lo obtenemos realizando los **mismos pasos** que **complemento a 1** hasta **antes** de agregar el **bit de signo**, es decir, primero **escribimos** el número en binario con **7 bits**:

$$0010100$$

Representación complemento a 2

- **Intercambiamos** ceros por unos y unos por ceros:

0	0	1	0	1	0	0
↓	↓	↓	↓	↓	↓	↓
1	1	0	1	0	1	1

- Al resultado le **sumamos** la **unidad**:

	1	1	0	1	0	1	1
+							1
	1	1	0	1	1	0	0

Representación complemento a 2

- Finalmente, **agregamos** el bit de signo:

$$11101100 = -20$$

- Podemos **comprobar** el resultado de la siguiente manera:

$$\begin{aligned} -20 &= -1(2^7) + 1(2^6) + 1(2^5) + 1(2^3) + 1(2^2) \\ &= -128 + 64 + 32 + 8 + 4 \\ &= -20 \end{aligned}$$

Ejercicios

Represente los siguientes números en formato **signo-magnitud**, **complemento a 1** y **complemento a 2** con 8 bits:

- ± 10
- ± 33
- ± 105
- ± 127

